Escape analysis

When the JIT can make an allocation disappear

07

What is escape analysis?

A C2 JIT check: after inlining, can anyone outside the method see the new object?

- **No**: the allocation is removed; its fields become locals (scalar replacement).
- **Yes**: it is allocated on the heap as usual.
- On by default; `-XX:-DoEscapeAnalysis` turns it off.

The decision

After inlining, C2 asks: can anyone outside this method see the new object?



Escapes = stored in a field, returned, or passed to code that was not inlined. Turn it off: -XX:-DoEscapeAnalysis

- Works only on compiled, inlined code, after warm-up.

In code

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return Iterate.anySatisfy(list, each -> each.equals(target));
}
```

- After inlining, the lambda and the loop stay local: 0 B allocated.
- The JDK stream escapes into its own stages: 160 to 224 B per call either way.

References

- Java HotSpot VM Performance Enhancements (Java SE 25): Escape Analysis
- Escape Analysis in the HotSpot JIT Compiler (Oracle Java Magazine)
- The `java` command (Java SE 25): `-XX:+DoEscapeAnalysis`